Main Difference Between a Process and a Thread in Linux

A **process** is an independent execution unit with its own address space, file descriptors, and system resources. A **thread** is a lightweight execution unit within a process that shares the same memory and resources as other threads in the same process.

Key Differences:

Feature	Process	Thread
Memory Space	Each process has its own memory space (address space).	Threads share the same memory space within a process.
Communication	Inter-process communication (IPC) mechanisms (pipes, shared memory, message queues) are needed.	Threads communicate easily via shared memory (since they belong to the same process).
Resource Overhead	More overhead due to separate memory management, file descriptors, etc.	Less overhead since they share resources.
Context Switching	More expensive due to memory and resource switching.	Faster, as threads share resources.
Isolation	Processes are independent; a crash in one does not affect others.	A crash in one thread can bring down the whole process.
Creation Time	Creating a new process is slower (`fork()` involves copying address space).	Creating a new thread is faster (`pthread_create()` uses shared memory).
Synchronization	More difficult due to separate address spaces.	Easier but requires careful synchronization (e.g., mutexes, semaphores) to avoid race conditions.

Pros and Cons of Processes and Threads

Processes:

✅ **Pros**

- Strong isolation (one process cannot easily corrupt another).
- More robust against crashes (a failed process doesn't affect others).
- Better security due to separate memory space.

❌ **Cons**

- More resource-intensive (each process has its own memory and resources).
- Slower communication due to IPC mechanisms.
- More overhead in context switching.

Threads:

✅ **Pros**

- Faster context switching.
- Easier communication (shared memory).
- More efficient resource usage.

❌ **Cons**

- Lack of isolation (a crashing thread can bring down the whole process).
- Requires synchronization mechanisms to prevent race conditions and deadlocks.
- Debugging is harder due to shared memory and race conditions.

When to Use Which?

- Use **processes** when **isolation and stability** are priorities (e.g., running independent services, microservices).
- Use **threads** when **performance and efficiency** matter, especially for tasks that share data (e.g., web servers, parallel computing).

What is Shared and What is Not in Threads (Linux/POSIX Threads)

When multiple threads exist within the same process, they share some resources while keeping others private.

Shared Among Threads (Common to All Threads in a Process)

✅ **Memory (Address Space)**: All threads share the same heap and global/static variables.

✅ **File Descriptors**: If one thread opens a file, others can access it using the same descriptor.

✅ **Code Section (Text Segment)**: All threads execute the same program instructions.

✅ **Global and Static Variables**: Since they are in the shared memory space, all threads can read/write them.

✅ **Heap Memory**: Dynamically allocated memory (`malloc`, `new`) is accessible to all threads.

✅ **Process ID (PID)**: Threads have the same PID but different Thread IDs (TIDs).

✅ **Signal Handlers**: Signals sent to the process affect all threads unless explicitly handled.

Not Shared (Each Thread Has Its Own Copy)

❌ **Stack Memory**: Each thread has its own stack for local variables and function calls.

❌ **Registers & Program Counter (Instruction Pointer)**: Each thread has its own execution context.

❌ **Thread-Specific Data (TLS - Thread Local Storage)**: Each thread can have its own separate data that isn't shared.

❌ **Thread ID (TID)**: Each thread has a unique ID even though they share the same process ID.

Why is this important?

- **Shared resources** (e.g., heap, global variables) require synchronization mechanisms like mutexes to prevent race conditions.
- **Private stack memory** ensures function calls and local variables are thread-safe.
- **File descriptors being shared** means that operations on a file by one thread affect all threads.

Thread Scheduling vs. Process Scheduling in Linux

Both **process scheduling** and **thread scheduling** are handled by the Linux kernel, but they differ in scope and complexity.

1. Process Scheduling

- The **Linux scheduler** schedules processes using **CFS (Completely Fair Scheduler)** or real-time policies.
- Each process is treated as an independent entity and has its own address space.
- The kernel assigns CPU time to a process based on its priority, policy, and resource usage.
- When a **process switch (context switch)** happens, the kernel must:
  - Save the process state (registers, memory mappings, etc.).
  - Load the new process state.
  - Switch the address space (TLB flush).
- **Processes can run in parallel on multiple CPUs (multi-processing).**

✅ **Pros**: Strong isolation, good for multi-programming.

❌ **Cons**: Context switching is expensive (requires saving/restoring memory and CPU state).

2. Thread Scheduling

- Threads **belong to a process** and share the same address space.
- The Linux kernel treats threads as **lightweight processes (LWPs)** using the `task_struct` data structure.
- Since threads share the same address space, **context switching is faster** (no need to switch memory maps).
- **Thread scheduling is handled the same way as process scheduling**, but with a key difference:

    - Threads within the same process can be scheduled **independently** on different CPUs.
    - Switching between threads is cheaper than switching between processes.
- Linux allows **both kernel-level and user-level threads**:
    - **Kernel-level threads** (e.g., `pthread` with `NPTL`) are scheduled by the OS just like processes.
    - **User-level threads** (e.g., green threads) are scheduled by a user-space library, not the kernel.

✅ **Pros**: Fast context switching, efficient for multi-threaded programs.

❌ **Cons**: Threads can interfere with each other (no memory isolation), race conditions.

Key Differences

Feature	Process Scheduling	Thread Scheduling
Context Switching Overhead	High (switching address space, memory mapping)	Low (only registers and stack switching)
Memory Isolation	Processes are isolated from each other	Threads share memory (risk of race conditions)
Parallel Execution	Multi-process programs can run on different CPUs	Multi-threaded programs can run on different CPUs
Scheduling Entity	The entire process is scheduled	Individual threads are scheduled independently
Communication	Inter-process communication (IPC) required	Threads communicate easily via shared memory
Execution Speed	Slower due to higher overhead	Faster due to lower overhead

When to Use What?

- Use **processes** when you need **strong isolation** (e.g., running separate services).
- Use **threads** when you need **fast context switching and shared memory** (e.g., web servers, real-time applications).

Creating, Getting Thread IDs, and Exiting Threads

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// Thread function
void* thread_function(void* arg) {
    printf("Thread %ld: Started\n", pthread_self());
    sleep(2);
    printf("Thread %ld: Exiting\n", pthread_self());
```

```
    pthread_exit(NULL);
}

int main() {
    pthread_t thread1, thread2;

    // Create two threads
    if (pthread_create(&thread1, NULL, thread_function, NULL)
!= 0) {
        perror("Failed to create thread 1");
        return EXIT_FAILURE;
    }
    if (pthread_create(&thread2, NULL, thread_function, NULL)
!= 0) {
        perror("Failed to create thread 2");
        return EXIT_FAILURE;
    }

    printf("Main Thread ID: %ld\n", pthread_self());

    // Wait for threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Main Thread: All threads completed.\n");
    return EXIT_SUCCESS;
}
```

Understanding `pthread_create()`

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

Parameter	Description
`thread`	A pointer to a `pthread_t` variable that will hold the thread ID.
`attr`	Thread attributes (can be `NULL` for default attributes).
`start_routine`	Function pointer to the thread's main function (must take `void*` and return `void*`).
`arg`	Pointer to the argument passed to the thread function (must be `void*`).

Return Values of `pthread_create()`

The function **`pthread_create()`** returns an **integer** that indicates whether the thread creation was **successful** or **failed**.

Syntax:

```
int pthread_create(pthread_t *thread, const pthread_attr_t
*attr, void *(*start_routine) (void *), void *arg);
```

Possible Return Values:

Return Value	Meaning
0	✅ **Success** – Thread created successfully.
`EAGAIN`	❌ **Not enough resources** to create a new thread.
`EINVAL`	❌ **Invalid settings in `attr`** (if a bad attribute is passed).
`EPERM`	❌ **Insufficient permissions** to set scheduling parameters in `attr`.